

Guía

de Programación Python

Programación
para todos.

Cesar Daniel Alayón Garavito

Introducción a la programación en Python

Introducción

En los últimos años la programación ha sido de los pilares fundamentales para el avance tecnológico, debido a que esta se encuentra en la mayor parte de los aparatos electrónicos.

Los lenguajes de programación, se han simplificado y optimizado con el pasar del tiempo, esto hace que se reduzca el código que se debe escribir para crear alguna funcionalidad específica, gracias a esto han surgido nuevos lenguajes con una sintaxis sencilla y fácil de aprender, pero a pesar de tener una curva de aprendizaje sencilla, el software que se puede crear con estos, es demasiado potente y optimizado, un gran ejemplo de esto es el lenguaje de programación Python, ya que es uno de los lenguajes más sencillos de aprender y con el cual se pueden realizar grandes avances tecnológicos, uno de estos es la inteligencia artificial (**IA**) y el Machine Learning.

La IA se ha comenzado a integrar, en casi toda la tecnología, encontramos carros que se manejan y/o se estacionan solos, celulares que cuentan con IA integrada en sus cámaras, bots que se encargan de responder los mensajes de clientes de una empresa de forma automática, y en más funcionalidades de diferentes aparatos electrónicos, por lo cual, desde hace unos años los programadores que se especializan en IA, son muy pedidos en el mundo laboral, y si se tiene en cuenta que la inteligencia artificial y sus ramas se programan normalmente en Python, este se vuelve uno de los lenguajes más solicitados y mejor pagados del mundo.

A pesar de esto, Python no solo es pedido para inteligencia artificial ya que este cuenta con muchas más funcionalidades, por ejemplo, para crear páginas web, entre otras.

Debido a esto se creó esta guía, en la cual, se va a enseñar lo básico de la programación en Python, el objetivo es que, al finalizar de la guía, puedas decidir qué es lo siguiente que quieres estudiar y no empezar desde 0, ya sea diseño web, programación competitiva, o algo más complejo como desarrollar de IA.

Historia

Guido Van Rossum es un científico en computación que comenzó a crear el lenguaje Python como un pasatiempo, el nombre del lenguaje surgió debido a que a Guido le gustaba el grupo Monty Python, él se basó en el lenguaje de programación ABC el cual fue creado por la compañía en la cual él trabajaba, la primera aparición pública del lenguaje se dio en el año 1991 con su versión 0.9.0 el cual solo contaba con: clases con herencias, funciones, tipos modulares y el manejo de excepciones. La versión 1.0 salió ese mismo año, está ya contaba con algunas de las funciones más usadas.

La versión 2.0 inicio con las características más importantes del lenguaje como lo son las listas. En el año 2001 se creó la Python Software Foundation, siendo esta la propietaria actual de todo el código y su respectiva documentación.

La versión 3.0 se publicó en el año 2008, esta se creó principalmente para corregir algunos errores que surgieron en la anterior versión.

Usos

Con Python se puede crear cualquier tipo de software, pero los puntos fuertes de Python es el desarrollo web con frameworks como Django, Inteligencia artificial (**IA**), Machine Learning, Big Data, Data science.

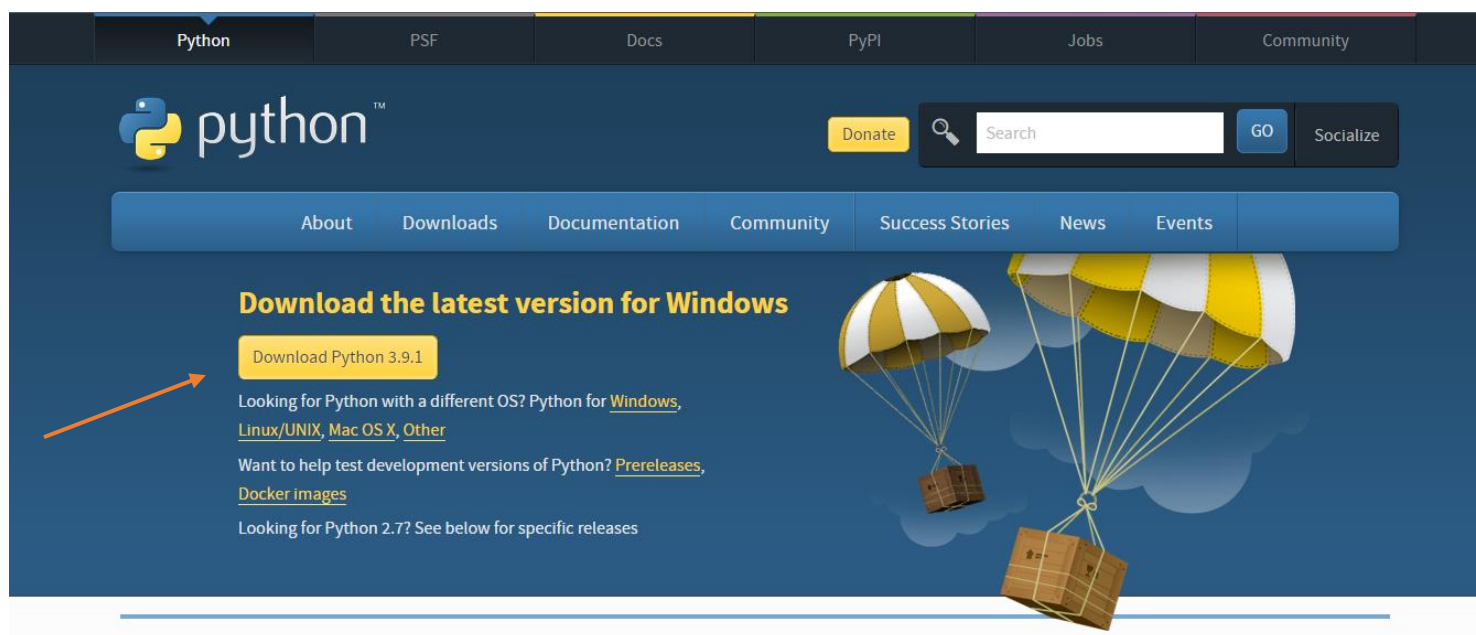
- **Desarrollo web:** El framework Django permite que se pueda crear una página web dinámica y segura con muy pocas líneas de código, lo cual es una ventaja para los desarrolladores web cuando se está en proyectos de páginas webs complejas, ya que al trabajar con Django la creación es mucho más eficiente, pero sobre todo sencilla.
- **IA:** Una de las razones que hace que Python se use en la IA es el hecho de que este es de escritura rápida, escalable y de código abierto. Se puede plasmar ideas complejas con unas pocas líneas de código, y se tienen librerías como keras y tensorFlow que tienen mucha información sobre el aprendizaje automatizado.
- **Big Data:** Python cuenta con librerías como Pydoop que facilita el análisis de datos y la extracción de información esto ayuda mucho a los profesionales por lo cual este se utiliza con demasiada frecuencia en el análisis y gestión de datos.
- **Data Science:** Muchos programadores se han pasado de lenguajes como MATLAB a Python debido a que en este existen motores numéricos como Pandas y NumPy, este se ocupa de datos tubulares, matriciales y estadísticos y los visualiza con librerías como Matplotlib y Seaborn.

Características

- **Lenguaje de alto nivel:** Este tipo de lenguajes son los que se acercan al lenguaje humano, para que este, sea capaz de comprenderlo.
- **Lenguaje interpretado:** En Python no necesitas un compilador para poder ejecutarlo, en cambio, necesita de un intérprete, la diferencia más marcada entre un compilador y un intérprete es que el compilador genera un archivo `.exe` para ejecutar el programa mientras que el intérprete lo ejecuta directamente en el IDE.
- **Tipado dinámico:** A las variables que se definen en Python no es necesario definirle un tipo, debido a que este se define automáticamente a la hora de ejecutar el programa.
- **Multiplataforma:** Los programas que se realizan en Python se pueden ejecutar en diferentes sistemas operativos como: Windows, MacOS, GNU/Linux, entre otros.
- **Multiparadigma:** Se pueden manejar diferentes técnicas a la hora de programar, esto facilita la elección de un paradigma (Técnicas) para la solución de un problema.

Descarga e instalación

Windows 8 o superior: Ingresar a la página www.python.org/downloads/ en el cual se encuentra la última versión del lenguaje; al presionar el botón se descargará automáticamente el instalador:



Para Windows 7 o un sistema operativo anterior, es necesario tener en presente que la versión de Python 3.9+ no funciona, por lo cual, se debe instalar Python 3.8.*, para esto, en la misma página se encuentra la siguiente sección:

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.8.7	Dec. 21, 2020	Download	Release Notes
Python 3.9.1	Dec. 7, 2020	Download	Release Notes
Python 3.9.0	Oct. 5, 2020	Download	Release Notes
Python 3.8.6	Sept. 24, 2020	Download	Release Notes
Python 3.5.10	Sept. 5, 2020	Download	Release Notes
Python 3.7.9	Aug. 17, 2020	Download	Release Notes
Python 3.6.12	Aug. 17, 2020	Download	Release Notes
Python 3.8.5	July 20, 2020	Download	Release Notes

[View older releases](#)

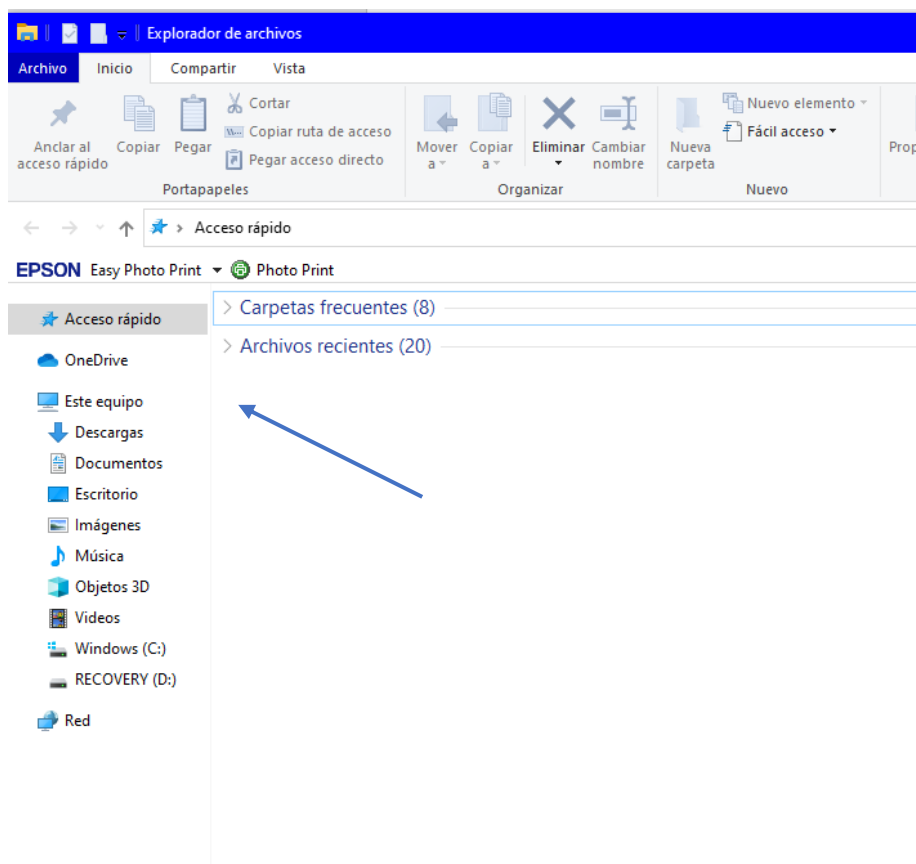
A blue arrow points to the 'Download' link for Python 3.8.7.

Se elige la versión de preferencia y se presiona el botón del centro, esto abrirá otra página, en la cual se aparecen los instaladores:

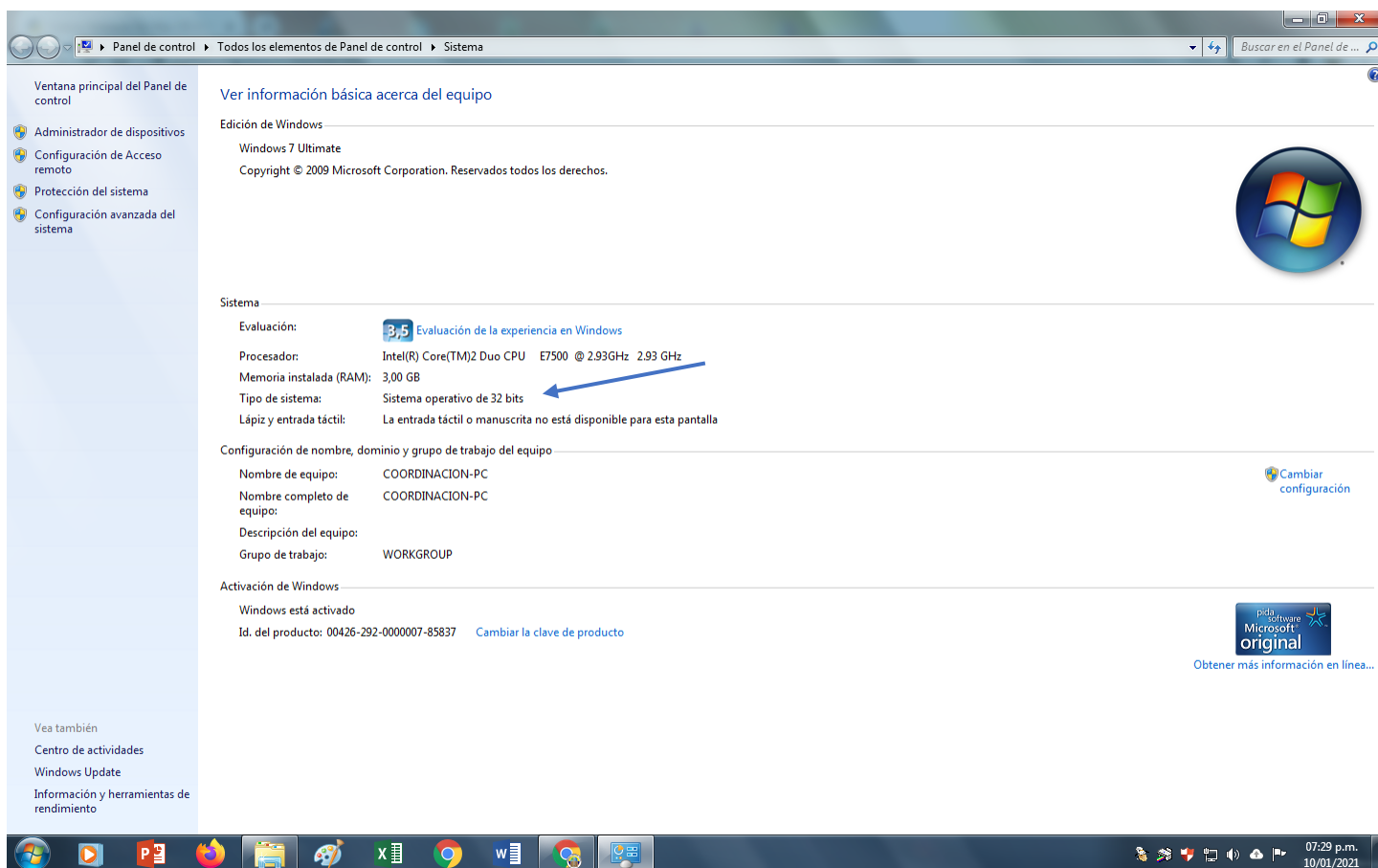
Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		e1f40f4fc9ccc781fcbf8d4e86c46660	24468684	SIG
XZ compressed source tarball	Source release		60fe018fffc7f33818e6c340d29e2db9	18261096	SIG
macOS 64-bit Intel installer	Mac OS X	for macOS 10.9 and later	3f609e58e06685f27ff3306bbcae6565	29801336	SIG
Windows embeddable package (32-bit)	Windows		efbe9f5f3a6f166c7c9b7dbbbe2cb24	7328313	SIG
Windows embeddable package (64-bit)	Windows		61db96411fc00aea8a06e7e25cab2df7	8190247	SIG
Windows help file	Windows		8d59fd3d833e969af23b212537a27c15	8534307	SIG
Windows installer (32-bit)	Windows		ed99dc2ec9057a60ca3591ccce29e9e4	27064968	SIG
Windows installer (64-bit)	Windows	Recommended	325ec7acd0e319963b505aea877a23a4	28151648	SIG

Las opciones que se pueden descargar son las dos últimas, pero es necesario saber cuál de las dos es compatible para esto se abre cualquier carpeta y hacer click derecho donde dice “Este equipo”:



Luego en el menú que sale, elegir propiedades, se abre la siguiente ventana:

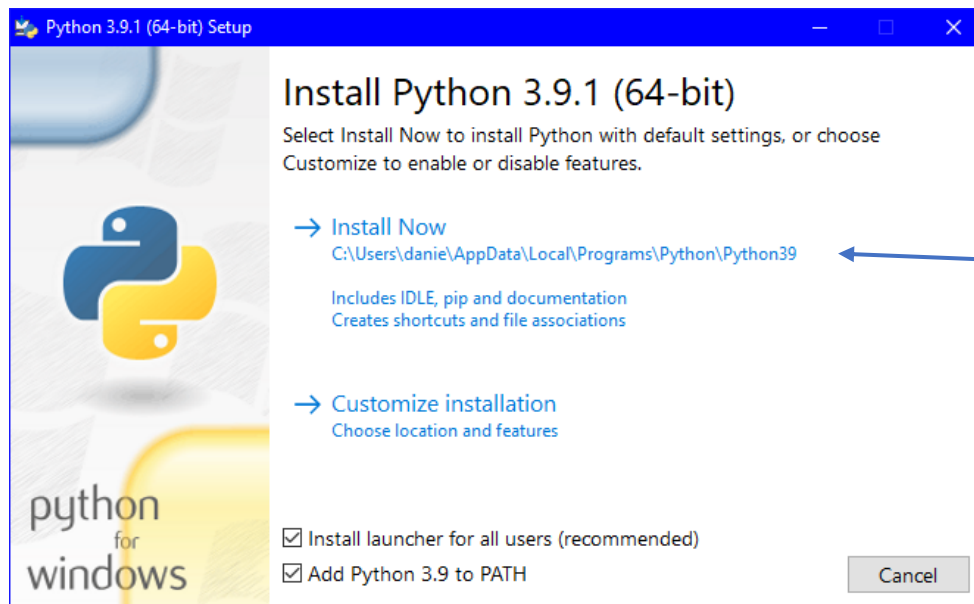
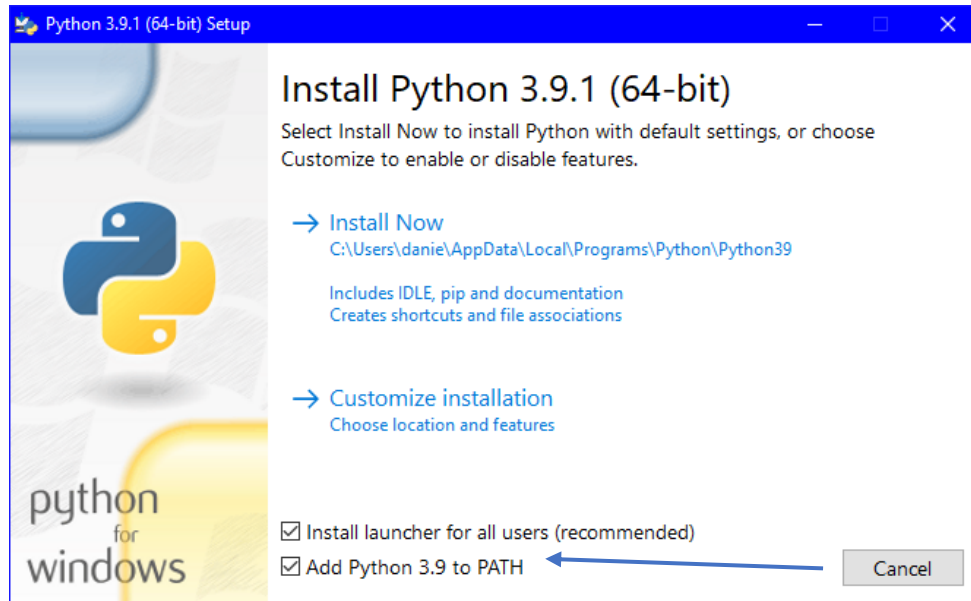


El número que está señalado indica la versión que se debe descargar ya sea 32 bits o 64 bits.

Por último, el link para descargar Python en el sistema operativo MacOS es el siguiente: www.python.org/downloads/mac-osx/

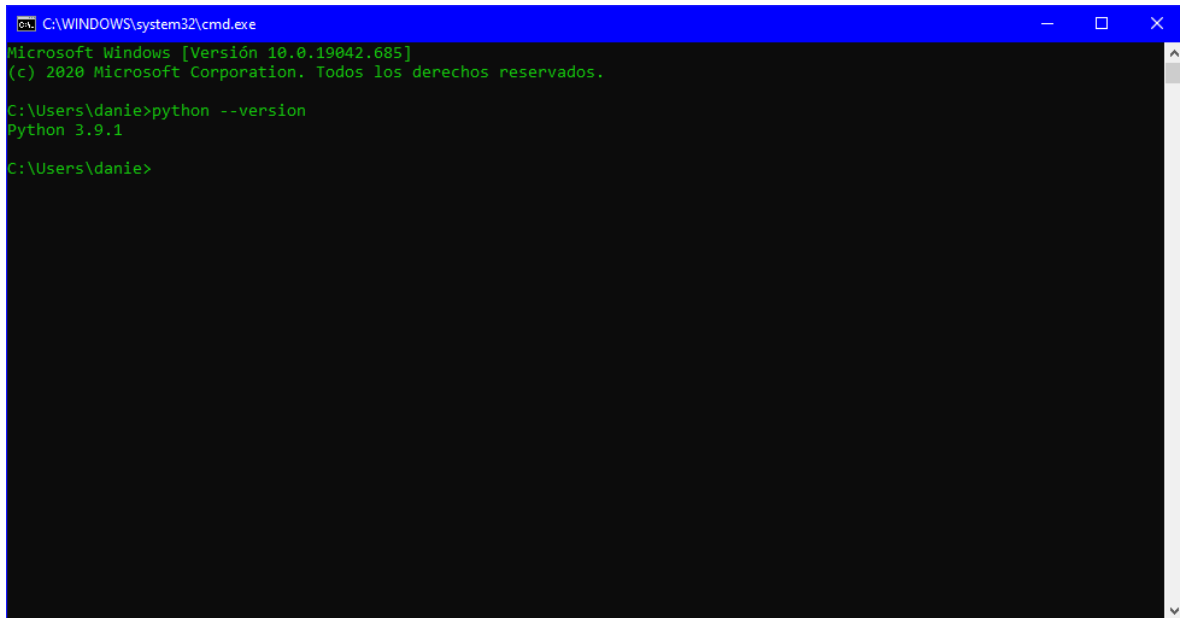
Al ejecutar el instalador lo más importante es que está marcada la opción **Add Python 3.X to PATH**, ya que esto le permitirá a Python ejecutarse en la consola, por lo cual se podrán instalar más características cuando se necesiten; es importante tener en cuenta que en algunos sistemas operativos el paso en el que

se encuentra esta casilla puede variar, para comenzar la instalación se debe presionar donde dice **Install Now**.



Una vez terminada la instalación se debe reiniciar el computador.

Para verificar que quedó correctamente instalado se puede abrir la terminal del sistema operativo y poner el comando: `python --version`



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Versión 10.0.19042.685]
(c) 2020 Microsoft Corporation. Todos los derechos reservados.

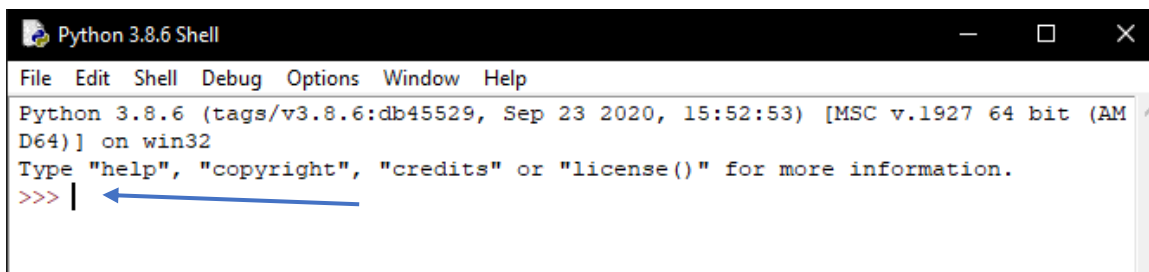
C:\Users\danie>python --version
Python 3.9.1

C:\Users\danie>
```

Esto mostrará la versión de Python que está instalada.

Para abrir Python se debe abrir la aplicación IDLE, la cual ya debe estar instalada en el computador.

Cuando se abre el IDLE DE Python encontraremos el PROMPT que indica donde se deben comenzar a digitar las líneas de código



```
Python 3.8.6 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6 (tags/v3.8.6:db45529, Sep 23 2020, 15:52:53) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Impresión

La primera línea será para imprimir el texto *Hola mundo*.

Cuando se quiere imprimir en Python se utiliza **print()**, cuando es un texto este se debe poner dentro de comillas ya sean dobles o sencillas, esto último no afectará el resultado de la impresión, cuando ya se tenga el código se debe presionar la tecla enter para ejecutarla

```
>>> print("Hola mundo")
Hola mundo
>>> print('Hola mundo')
Hola mundo
>>> |
```

El problema de utilizar el PROMPT es que no se puede editar el código que se escribió antes, por lo cual, si hay un error, corregirlo será algo complicado, lo mejor es crear un archivo Python. Para crearlo se debe presionar la combinación de teclas **(CTRL+N en el caso de Windows o CMD+N en Mac)** esto se realiza teniendo el PROMPT abierto y cuando se ejecute el comando se abrirá otra ventana en la cual se va a copiar la misma línea de código para imprimir hola mundo; para ejecutar este archivo se debe presionar la tecla **F5**, en ese momento se solicitará guardar el archivo, una vez guardado se ejecutará automáticamente en el PROMPT.



Variables y tipos de datos

Uno de los elementos más importantes a la hora de programar son las variables, las cuales, son un espacio reservado en la memoria en la que se almacena un dato.

Existen diferentes tipos de datos que se pueden almacenar en una variable:

Variable	Tipo	Tamaño
Int	Numérico (Entero)	Infinito
Float	Numérico (Decimal)	Infinito
Char	Carácter	Infinito
String	Cadena de caracteres	Infinito
Bool	True o False	No aplica

Tabla 1 – Creación Propia

Como se puede ver en la tabla 1, el tamaño de las variables en Python siempre es infinito, por lo cual el tamaño de los datos que se van a ingresar es algo que se puede ignorar, se debe tener en cuenta que al ser un lenguaje de tipado dinámico

no se tiene que especificar qué tipo de dato se está usando, ya que Python es un lenguaje de tipado dinámico, esto hace que Python asigne

Para definir una variable se debe seguir la siguiente estructura: **nombre_de_variable = dato;** para buenas prácticas es necesario que en el nombre de la variable se especifique para que se va a usar, y así facilitar la lectura del código, esto puede generar que algunos nombres contengan más de una palabra, lo recomendable en estos casos es separar cada palabra con un guion bajo “_”, también se debe tener en cuenta que las variables son case sensitive, por lo cual se debe llamar la variable con exactamente el mismo nombre con el que se creó.

Como se pudo ver, para imprimir una variable también se utiliza el **print()**, pero en este caso, no se ponen las comillas, solo se debe escribir el nombre de la variable que se quiera imprimir.

```
1 numero_entero = 2
2 numero_decimal = 3.5
3 caracter = 'a'
4 cadena_de_caracteres = "Hola Mundo"
5
6 print(numero_entero)
7 print(cadena_de_caracteres)
```

Cuando se requiere imprimir una variable acompañado de algún texto, se deben separar por medio de una coma, por ejemplo:

```
1 a = 50
2 b = 30
3 print("El primer numero es",a,"y el segundo",b)
```

Lectura

Los datos que están dentro de las variables se han definido en propio código, lo cual hace que para cambiar un dato se deba hacer desde el propio código, pero por comodidad es mejor leer el dato requerido desde la consola, para esto existe una función que nos permite leer los datos desde el PROMPT: **input()** , esta no solo tiene la función de leer los datos , también permite imprimir un texto para especificar el dato que se está pidiendo, el cual debe ir dentro de los paréntesis que se usan al definir el input, se debe tener en cuenta que al ser un string también debe estar dentro de comillas, este texto no es obligatorio, por ejemplo:

```
1 a = input('Inserta un numero: ')
2
3 print(a)
```

Por defecto la función va a leer el dato ingresado como string, por lo cual, si se quiere leer un dato como un número, se tiene que especificar de la siguiente manera:

```
1 a = int(input('Inserta un numero: '))
2
3 print(a)
```

Esto no funciona solo para datos de tipo **Int**, por lo cual esta conversión se puede aplicar para cualquier tipo de dato.

Operadores

Los operadores tienen diferentes funciones y se usan en todos los códigos, sin importar si es un código sencillo o algo más avanzado los operadores se pueden dividir en 5 categorías según la función que cumplen dentro de un código: Aritméticos, de comparación, lógicos, de asignación y operadores especiales:

Aritméticos	
Suma	+
Resta	-
Multiplicación	*
División	/
División entera	//
Potencia	**
Modulo	%

Dentro de los aritméticos se encuentran dos operadores para división, la diferencia entre estos, radica en que la división entera quita los decimales. El módulo es el encargado de mostrar el residuo que queda al dividir, por ejemplo, esto nos sirve

para saber si un número es divisible entre otro, ya que en ese caso el residuo del módulo entre estos será igual a cero.

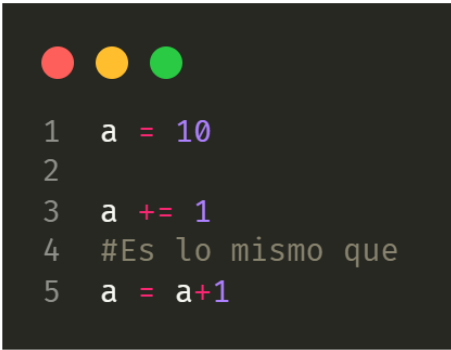
Comparación	
Igual que	==
Diferente que	!=
Mayor que	>
Menor que	<
Mayor o igual que	>=
Menor o igual que	<=

Los operadores de comparación permiten comparar diferentes datos ya sean anteriormente guardados en variables o definidos solo para la comparación, estos operadores nos lanzan valores de tipo booleano, por ejemplo:

```
1 a = 10 == 10
2 b = 10 == 11
3 print(a)
4 print(b)
```


Asignación	
Igual	=
Incremento	+=
Decremento	-=
Multiplicación	*=
División	/=
Modulo	%=
Potencia	**=
División Entera	//=

Los operadores de asignación se encargan de darle un valor a una variable, a partir del operador incremento hasta división entera, se encargan de hacer la operación respectiva sin tener que llamar otra vez a la variable, por ejemplo:



```
1  a = 10
2
3  a += 1
4  #Es lo mismo que
5  a = a+1
```

Dentro de los operadores lógicos encontramos: **AND, OR, NOT**; Estos nos ayudan a comparar más de dos valores o condicionales.

También encontramos los operadores especiales se encargan de revisar si un dato en concreto se encuentra en una **colección**: IS, IS NOT, IN, NOT IN.

Estructuras de datos

Listas

La lista, es una estructura de datos que permite almacenar varios valores que tengan alguna característica en común y que se quiera mantenerlos en conjunto; para definir una lista se deben poner los datos dentro de corchetes, separándolos por comas, por ejemplo:



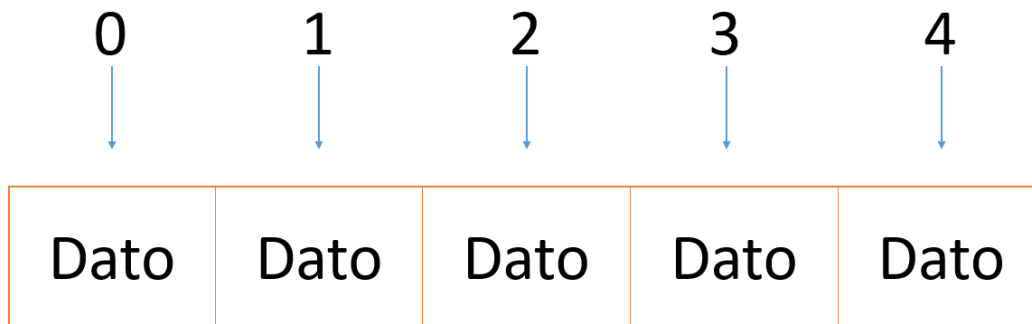
```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
```

Las listas al ser heterogéneas, permiten almacenar datos de diferente tipo, por ejemplo:



```
1 lista = ["Hola Mundo", 200, True, 10.5]
```

Cada elemento en una lista tiene una posición específica, la cual está dada por un índice que comienza desde 0:



Los elementos de una lista se pueden llamar teniendo en cuenta el índice(posición) en la cual está, por ejemplo:



```
1 estudiantes = ["Daniel", "Edwin", "Julian"]
2 print(estudiantes[0])
3 print(estudiantes[1])
4 print(estudiantes[2])
5
6 '''
7 El resultado esperado es:
8 Daniel
9 Julian
10 Edwin
11
12 '''
```

A través de los índices se pueden modificar los datos de una lista, para esto se debe poner el nombre de la lista, y el índice del elemento que se quiere modificar, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  print('Valor anterior:', estudiantes[0])
3  estudiantes[0] = "Samuel"
4  print('Nuevo valor', estudiantes[0])
5
6  '''
7  Resultado:
8  Valor anterior: Daniel
9  Nuevo valor: Samuel
10
11  '''
```

Un dato que se debe tener en cuenta a la hora de manipular una lista es la cantidad de datos que están en esta, la función **len()** permite saber el largo que tiene una lista, para usar la función **len()** se debe pasar como parámetro el nombre de la lista, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  largo_lista = len(estudiantes)
3  print('En la lista hay', largo_lista, 'datos')
4
5  '''
6  Resultado:
7  En la lista hay 3 datos
8  '''
```

A continuación, se verán los métodos que tienen integrados las listas:

- **Append():** Agrega un elemento al final de la lista:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  estudiantes.append("Valentina")
3  print(estudiantes)
4  #Resultado: ["Daniel", "Edwin", "Julian", "Valentina"]
5
```

- **Count():** Este método permite contar cuantas veces aparece un dato en una lista, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  contar = estudiantes.count("Daniel")
3  print(contar)
4  #Resultado: 1
```

- **Index():** Busca un elemento en la lista y muestra el índice en el que se encuentra, si el elemento esta varias veces en la lista devuelve la primera posición en la que el elemento se encuentra, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  print(estudiantes.index("Edwin"))
3  #Resultado: 1
```

- **Insert():** Se encarga de insertar un elemento en el índice que se le especifique, para esto a la hora de llamar a la función se debe pasar como parámetro el índice seguido del elemento que se quiere ingresar, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  estudiantes.insert(1, "Alejandro")
3  print(estudiantes)
4  #Resultado ["Daniel", "Alejandro", "Edwin", "Julian"]
```

- **Pop():** Devuelve el último elemento de la lista y lo elimina de la lista, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  print('Elemento Eliminado:', estudiantes.pop())
3  print(estudiantes)
4  #Resultado ["Daniel", "Edwin"]
```

- **Remove():** Busca el elemento que se le pasa por parámetro y borra su primera aparición, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian", "Daniel"]
2  print('Elemento Eliminado:', estudiantes.remove("Daniel"))
3  print(estudiantes)
4  #Resultado ["Edwin", "Julian", "Daniel"]
```

- **Reverse():** Esta función invierte el orden de los elementos en una lista, por ejemplo:

```
1  estudiantes = ["Daniel", "Edwin", "Julian"]
2  estudiantes.reverse()
3  print(estudiantes)
4  #Resultado ["Julian", "Edwin", "Daniel"]
```

- **Sort():** Esta función ordena los elementos que se encuentran en una lista, por defecto los ordena de menor a mayor, pero al pasarle el parámetro reverse con valor en True los ordena de mayor a menor, por ejemplo:

```
1  lista = [4,6,1,3,2,5]
2  lista.sort()
3  print(lista)
4  lista.sort(reverse=True)
5  print(lista)
6  '''
7  Resultado:
8  [1, 2, 3, 4, 5, 6]
9  [6, 5, 4, 3, 2, 1]
10 '''
```

Tuplas

Una tupla es una lista inmutable, es decir, el contenido de la tupla no se puede modificar después de su creación. Para crear una tupla se deben poner paréntesis y separar los datos con comas, por ejemplo:

```
1 tupla = (1, "Hola", 'a', 10.5)
2 print(tupla)
```

Las tuplas permiten almacenar 0 o más datos, aun así, si se requiere almacenar solo un dato es necesario poner una coma al final, ya que si no se pone lo lee como si fuera una variable, por ejemplo:

```
1 tupla = ()
2 print(tupla)
3 tupla2 = ('Hola',)
4 print(tupla2)
5 '''
6 Resultado:
7 ()
8 (hola,)
9 '''
```


Diccionarios

Los diccionarios permiten almacenar cualquier tipo de dato, al igual que las listas, pero en los diccionarios a cada dato se le asigna una clave, para crear un diccionario se deben usar llaves, dentro de estos se insertan los datos junto a sus claves creando una asignación de estilo **clave: dato**, por ejemplo:

```
1  diccionario = {'Hola':'Hello', 'Mundo':'World'}
```

Las claves deben ser únicas para cada elemento, por lo cual en un diccionario no se van a encontrar claves repetidas.

Para acceder a un valor se pueden usar las claves, de este modo:

```
1  diccionario = {'Hola':'Hello', 'Mundo':'World'}
2  print(diccionario['Hola'])
3  print(diccionario['Mundo'])
4  '''
5  Resultado:
6  Hello
7  World
8  '''
```

Las claves pueden ser numéricas, por ejemplo:

```
1  diccionario = {1:'Hello', 2:'World'}
2  print(diccionario[1])
3  print(diccionario[2])
```

Los diccionarios no son ordenados, por lo cual, tanto los valores, como las claves se pueden utilizar en cualquier orden.

Para cambiar un dato se puede llamar a la clave respectiva, y asignarle el nuevo dato, por ejemplo:

```
1  diccionario = {'País Uno': 'Colombia', 'País Dos': 'Rusia'}
2  print(diccionario['País Dos'])
3  diccionario['País Dos'] = 'Mexico'
4  print(diccionario['País Dos'])
```

Si se quiere eliminar alguno de los datos dentro de un diccionario se utiliza la palabra reservada **del**, esta se debe poner antes de llamar al valor usando su la clave correspondiente, por ejemplo:

```
1  diccionario = {'País Uno': 'Colombia', 'País Dos': 'Rusia'}
2  print(diccionario)
3  del diccionario['País Dos']
4  print(diccionario)
```

Dentro de los diccionarios se pueden almacenar también elementos como listas, por ejemplo:

```
1  diccionario = {'Países': ['Colombia', 'Mexico', 'Argentina']}
2  print('Lista de países:', diccionario['países'])
```

A continuación, se verán los métodos que tienen integrados los diccionarios:

- **Dict():** Esta función revisa si es posible generar un diccionario con los datos que se le pasan como parámetro, de ser posible, genera automáticamente un diccionario el cual se puede almacenar en una variable, por ejemplo:

```
1 diccionario = dict(nombre = 'Daniel', apellido = 'Alayón', edad = 19)
2 print(diccionario)
```

- **Zip():** Recibe como parámetro dos objetos iterables, como una lista o un string, y a partir de estos genera un diccionario, los objetos que se le pasan como parámetro deben tener la misma longitud, para poder leerlos correctamente, la función **zip** se debe pasar como parámetro de la función **Dict()**, para poder generar un diccionario por ejemplo:

```
1 diccionario = dict(zip('abcd', [1,2,3,4]))
2 print(diccionario)
```

- **Keys():** Esta función devuelve solamente las claves que hay en el diccionario, por ejemplo:

```
1 diccionario = {'Países':['Colombia', 'Mexico', 'Argentina']}
2 print(diccionario.keys())
```

- **Values():** Esta función devuelve solamente los valores que hay en el diccionario, por ejemplo:

```
1 1 2 3  
1 diccionario = {'Países': ['Colombia', 'Mexico', 'Argentina']}  
2 print(diccionario.values())
```

- **Clear():** Elimina todos los elementos del diccionario dejándolo vacío, por ejemplo:

```
1 1 2 3  
1 diccionario = {"Países": ["Colombia", "Rusia", "Mexico"]}  
2 print(diccionario)  
3 diccionario.clear()  
4 print(diccionario)
```

- **Copy():** Crea una copia del diccionario en una variable nueva

```
1 1 2 3  
1 diccionario = {"Países": ["Colombia", "Rusia", "Mexico"]}  
2 copia_diccionario = diccionario.copy()  
3 print(copia_diccionario)
```

- **Get():** Esta función devuelve el valor de la clave que se le pasa como parámetro, si esta clave no existe en el diccionario devuelve un objeto none, por ejemplo:

```
1 1 2 3  
1 diccionario = {"Países": ["Colombia", "Rusia", "Mexico"]}  
2 resultado = diccionario.get("Países")  
3 print(copia_diccionario)
```

- **Pop():** Esta función se encarga de mostrar el valor de la clave que se le pasa por parámetro, y al hacerlo, también elimina el valor junto a su clave, por ejemplo:

```
1 diccionario = {"1":"uno", "2":"dos", "3":"tres", "4":"cuatro", "5":"cinco"}
2 print(diccionario)
3 diccionario.pop("5")
4 print(diccionario)
```

- **Setdefault():** Esta función tiene dos objetivos, el primero es imprimir un valor, pasándole como parámetro la clave y el segundo, es añadir un nuevo elemento en el diccionario, por ejemplo:

```
1 diccionario = {"1":"uno", "2":"dos", "3":"tres", "4":"cuatro", "5":"cinco"}
2 print(diccionario.setdefault("1"))
3 diccionario.setdefault("6", "seis")
4 print(diccionario)
```

- **Update():** Esta función recibe otro diccionario, si tiene claves iguales el valor es remplazado, si el segundo diccionario cuenta con objetos extra, estos son añadidos al diccionario principal, por ejemplo:

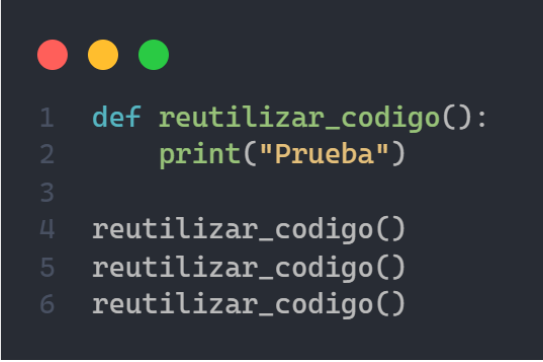
```
1 diccionario = {"1":"uno", "2":"dos", "3":"tres", "4":"cuatro"}
2 diccionario2 = {"1":"uno", "2":"dos", "3":"tres", "4":"cuatro", "5":"cinco"}
3 print(diccionario)
4 diccionario.update(diccionario2)
5 print(diccionario)
```

Funciones

Las funciones también conocidas como métodos, son líneas de código que se agrupan para que realicen una tarea específica, lo que permite esto es que el código este más ordenado, pero, la principal razón por la cual se utilizan las funciones, es que estas nos permiten reutilizar el código, esto debido a que, en algunos casos, se necesita usar las mismas líneas de código varias veces dentro de un mismo programa, lo cual puede extender bastante el código. La estructura de una función es:

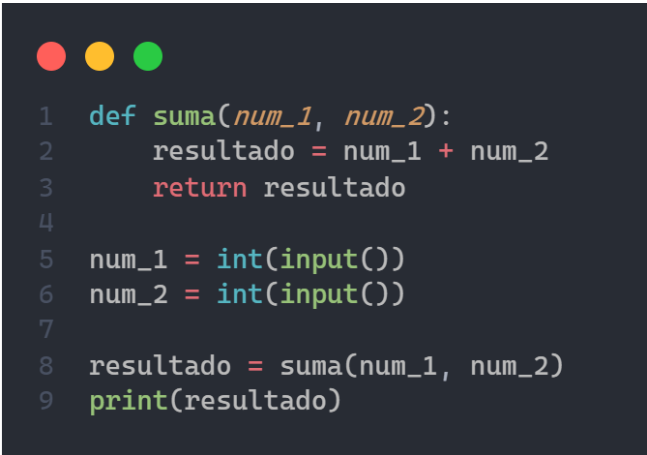
```
1 '''
2 La palabra def es reservada, se usa siempre que se crea una función.
3 Dentro de los paranteses se ponen los parametros que se necesiten.
4 '''
5 def nombre_funcion():
6     #Aquí van las instrucciones
7     a = 3
8     #Para devolver un valor se debe usar return
9     return a
10 '''
11 Para llamar la función se pone el nombre, más los paréntesis.
12 Si se retorna un valor como en este caso, se puede guardar en una variable pero
13 tambien se pueden utilizar directamente.
14 '''
15 numero = nombre_funcion()
16 print(numero)
```

Si se tiene una función que se quiera reutilizar lo único que se requiere es llamarla otra vez, es importante tener en cuenta que las veces que se puede reutilizar una función es ilimitada, y se puede utilizar en diferentes partes del código, un ejemplo sencillo es que se tenga un texto que se deba imprimir varias veces, si se quiere hacer por medio de una función sería algo así:



```
1 def reutilizar_codigo():
2     print("Prueba")
3
4 reutilizar_codigo()
5 reutilizar_codigo()
6 reutilizar_codigo()
```

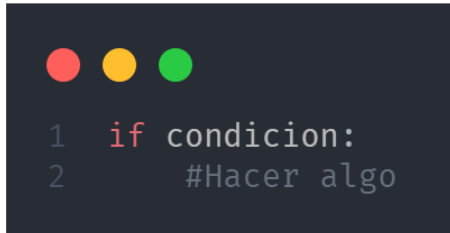
En ciertas ocasiones se necesita pasar un dato específico por una función, a estos datos se les conocen como parámetros, para definir un parámetro se escribe el nombre de la variable dentro de los paréntesis a la hora de crear la función, si se necesita más de un parámetro/variable, estos se deben separar por medio de una coma, por ejemplo:



```
1 def suma(num_1, num_2):
2     resultado = num_1 + num_2
3     return resultado
4
5 num_1 = int(input())
6 num_2 = int(input())
7
8 resultado = suma(num_1, num_2)
9 print(resultado)
```

Condicionales

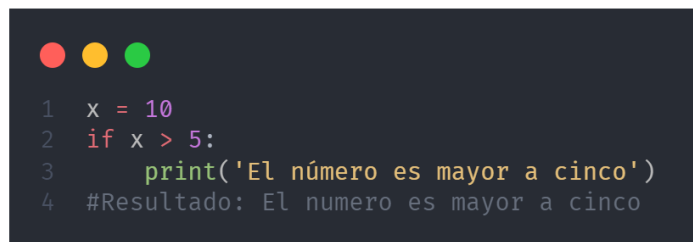
La condicional **if** permite ejecutar código solamente si se cumple una condición específica, su estructura es la siguiente:



```
1  if condicion:
2      #Hacer algo
```

Si la condición definida en el if, no se cumple, el código dentro de este no se ejecuta; se debe tener en cuenta que dentro de la condicional se usan los operadores de comparación,

por ejemplo:



```
1  x = 10
2  if x > 5:
3      print('El número es mayor a cinco')
4  #Resultado: El numero es mayor a cinco
```

El programa verifica si el número ingresado es mayor a 5, si el número es mayor, se ejecuta el código dentro de la condicional, en este caso imprime, [El número es mayor a cinco](#), pero si no se cumple, el código no ejecuta nada.

Dentro de la condicional if se pueden utilizar los operadores lógicos **AND** y **OR**, el operador **AND** permite ejecutar el código dentro de la condicional solo si se cumplen todas las condiciones que se le pasen al if, en cambio, el operador **OR**, ejecuta el código si se cumple alguna de las condiciones.

Si se requiere ejecutar unas instrucciones específicas cuando no se cumpla una condicional **if** se debe complementar con una condicional **else**, ya que esta permite ejecutar código únicamente si la condicional anterior es falsa, su estructura es la siguiente:



```
1  if condicion:
2      #Hacer algo si se cumple la condicion
3  else:
4      #Hacer algo si no se cumple la condicion anterior
```

Aplicando esta estructura al código para verificar si un número es mayor a 5, quedaría:



```
1  x = 2
2  if x > 5:
3      print('El número es mayor a cinco')
4  else:
5      print('El número es menor a cinco')
6  #Resultado: El numero es menor a cinco
```

Práctica: Crear un programa que pida la edad del usuario y si tiene una edad igual o mayor de 18 años permitir el acceso al sistema, si es menor, denegar el acceso.

La condicional `elif` es la unión de la condicional `else` junto a la condicional `if`, esto permite ejecutar instrucciones si se cumple lo siguiente:

- Ninguna de las condicionales anteriores fue verdadera
- Se cumple la condición que tiene definida

Su estructura es la siguiente:

```
1 if condicion:
2     #Hacer algo
3 elif condicion2:
4     '''
5     Hacer algo, si no se cumple la condicional anterior
6     y si se cumple la condicion
7     '''
8 else:
9     #Hacer algo si no se cumple ninguna de las condiciones anteriores
```

Por ejemplo, se quiere verificar si una persona es mayor de edad, y se quiere evitar que el usuario digite un número negativo, se haría de la siguiente manera:

```
1 edad = int(input('Digita tu edad: '))
2 if edad < 18 and edad >= 0:
3     print('Eres menor de edad')
4 elif edad >= 18:
5     print('Eres mayor de edad')
6 else:
7     print('Digita un numero positivo')
```

Un ejemplo en el que se puede utilizar todas las condicionales juntas es al hacer una calculadora, en la cual se le permite elegir al usuario la operación que quiere realizar:

```

1  #Pedir numeros al usuario
2  primer_numero = int(input('Digita un número entero: '))
3  segundo_numero = int(input('Digita otro número entero: '))
4  #Menu
5  print('Operaciones disponibles')
6  print('1) Suma')
7  print('2) Resta')
8  print('3) Multiplicacion')
9  #Lectura operacion
10 operacion = int(input('Digita el numero de la operacion a realizar: '))
11 #Condicionales
12 if operacion == 1:
13     print('La suma de los numeros es:', primer_numero + segundo_numero)
14 elif operacion == 2:
15     print('La resta de los numeros es:', primer_numero - segundo_numero)
16 elif operacion == 3:
17     print('La multiplicacion de los numeros es:', primer_numero * segundo_numero)
18 else:
19     print('El numero ingresado no corresponde a una operacion intente nuevamente')

```

Practica: Añadir al código anterior las operaciones: división, división entera, potencia.

Ciclos (Bucles)

Los ciclos, también conocidos como bucles, permiten ejecutar un conjunto de instrucciones, varias veces.

Ciclo while

La traducción de la palabra while es ‘mientras’, por lo cual este bucle se puede resumir en lo siguiente: “Mientras haya algo que hacer, hazlo”.

Su estructura es la siguiente:

```

1  while condicion:
2      #hacer algo

```

Por ejemplo, se quiere imprimir un texto, pero solamente mientras un número sea menor a 5:

```
1 num = 0
2 while num < 5:
3     print('El numero sigue siendo menor a cinco')
4     num += 1
```

En el anterior código si no se incrementara la variable `num` el ciclo se volvería infinito, esto debido a que siempre se está cumpliendo la condición, esto se puede verificar poniendo en la condición del while, el valor booleano `True`:

```
1 while True:
2     print('Ciclo Infinito')
```

Los ciclos permiten explorar de una manera sencilla las estructuras de datos, ya que, cada vez que se ejecute el bucle se aumenta el valor de una variable lo que permite ir cambiando el valor del índice, por ejemplo:

```
1 lista = ['Daniel Alayon', 'Edwin', 'Julian']
2 largo_lista = len(lista)
3
4 while largo_lista > 0:
5
6     largo_lista -= 1
7
8     print(lista[largo_lista])
```

Esto mostrará cada elemento que esta dentro de la lista, y esta programado de tal manera que si a la lista se le agrega un elemento más el código seguirá funcionando.

Con este método también se pueden agregar elementos a una lista, esto permite que el usuario decida cuantos elementos hay que agregar e ingresar los datos respectivos, se debe tener en cuenta que se usara el método Append perteneciente a las listas, por ejemplo:

```
1 lista = []
2 largo_lista = int(input('Ingresa la cantidad estudiantes: '))
3
4 while largo_lista > 0:
5     largo_lista -= 1
6     lista.append(input('Nombre del Estudiante: '))
7
8 print(lista)
```

Ciclo For

El ciclo for permite definir un rango, por ejemplo, del 0 al 15, en este caso el ciclo se ejecutará 15 veces.

La estructura del for es la siguiente:

Variable Inicio Fin Paso

↓ ↓ ↓ ↓

for **i** **in** **range**(0, 15, 1)

Es importante tener en cuenta varias cosas:

- La variable almacena el número en el cual está el ciclo en ese momento, por lo cual cada vez que se completa un ciclo, el valor de la variable va a aumentar.
- La variable está almacenada de manera local y por lo tanto se puede usar FUERA del bloque del ciclo.
- El número final del rango, siempre tiene un número de más, por lo cual si se imprime la variable esta solo llegará al número 14.
- El valor del inicio y del paso están definidos de manera predeterminada en 0 y respectivamente, por lo cual se puede definir solamente el número final del rango.
- Se puede utilizar un for dentro de otro pero el nombre de la variable debe ser diferente
- El paso es cuanto aumenta el valor del rango con respecto al anterior, por ejemplo, si se tiene un rango del 0 al 10 con paso ,1 el ciclo será:
0,1,2,3,4,5,6,7,8,9,10 pero si es el mismo rango con paso 2, el ciclo será:
0,2,4,6,8,10

Ejemplos:

```
1 for i in range(0, 15, 2):  
2     print(i)
```

```
1 for i in range(0, 15, 1):  
2     print(i)
```

El anterior código es lo mismo que:

```
1 for i in range(15):  
2     print(i)
```

El ciclo for sirve también para la manipulación de estructuras de datos, por ejemplo:

```
1 lista = []  
2 largo_lista = int(input('Cantidad de estudiantes a ingresar: '))  
3 for i in range(largo_lista):  
4     lista.append(input('Nombre del Estudiante'))
```

Ciclo for vs While

Aunque la función de los dos bucles es parecida, se encuentran diferencias que hacen más efectivos para ciertos códigos:

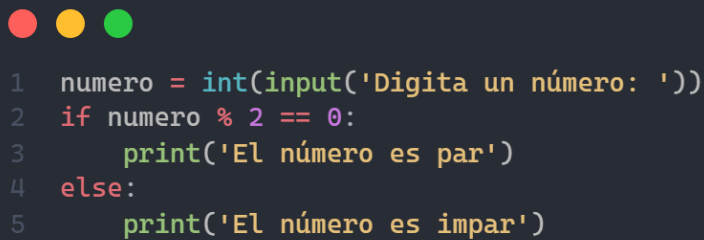
- El ciclo while acepta cualquier condición para la creación de un ciclo.
- El for solo genera un rango de números.

Debido a lo anterior el for es mejor cuando se van a manipular rangos de números, y el while cuando se debe crear un bucle usando cualquier otro tipo de condición.

Ejercicios con códigos:

A continuación, se mostrarán la solución a algunos ejercicios usando los temas vistos hasta el momento.

Verificar si el número ingresado es par:



```
1  numero = int(input('Digita un número: '))
2  if numero % 2 == 0:
3      print('El número es par')
4  else:
5      print('El número es impar')
```


Verificar si el número ingresado es primo:

```
1 numero = int(input('Digita un número: '))
2 verificacion = True
3
4 for i in range(2, numero):
5     if numero % i == 0:
6         verificacion = False
7
8 if verificacion == True:
9     print('El número es primo')
10 else:
11     print('El numero no es primo')
```

Imprimir los números pares entre 0 y x

```
1 x = int(input())
2 for i in range(0, x, 2):
3     print(i)
```

Verificar si un número está en una lista

```
1 lista = [5,3,50,4,6,8,12,23,31,24,64,41,18]
2 x = int(input('Digita un número: '))
3 if x in lista:
4     print('El número está en la lista')
5 else:
6     print('El número no está en la lista')
```

Programación Orientada a objetos

La programación orientada a objetos (POO) es un paradigma (estilo) de programación que permite pasar objetos de la vida real a código, la idea, es traspasar a código las características del objeto, tanto su aspecto físico como sus otras características, por ejemplo:

Un automóvil, tiene estados, propiedades y un comportamiento:

1. Estado: Parado, Circulando, etc.
2. Propiedades: color, peso, medidas
3. Comportamiento: Arrancar, frenar, acelerar, girar, etc.

Conceptos basicos dentro de la POO:

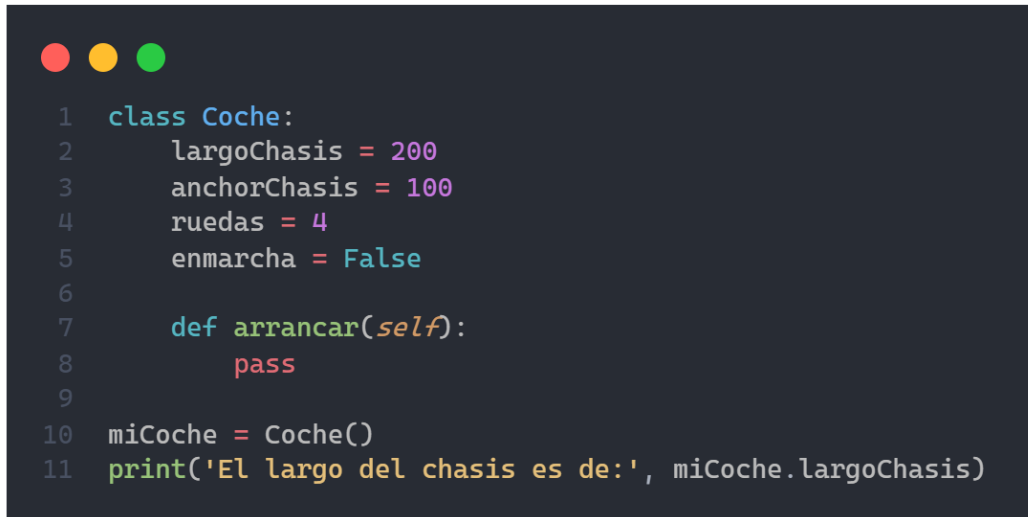
1. **Clase:** Donde se encuentran las características comunes entre un grupo de objetos
2. **Instancia:** objeto perteneciente a una clase
3. **Modularización:** Dividir un programa en módulos, cada módulo se puede compilar por separado, pero se pueden generar conexiones entre ellos.

Para definir una clase se debe usar la palabra reservada `class` seguido de su respectivo nombre, (Es recomendado, que el nombre de la clase tenga la primera letra en mayúscula esto para diferenciar entre funciones y clases).

Dentro de las clases se usa la palabra reservada `self`, la cual, funciona para llamar objetos o funciones, pertenecientes a la misma clase.

Para definir una característica se pueden utilizar variables, para definir el comportamiento se usan funciones y para definir el estado del objeto se pueden usar variables booleanas.

Ejemplo:



```
1 class Coche:
2     largoChasis = 200
3     anchoChasis = 100
4     ruedas = 4
5     enmarcha = False
6
7     def arrancar(self):
8         pass
9
10 miCoche = Coche()
11 print('El largo del chasis es de:', miCoche.largoChasis)
```

Cuando se define un objeto, se puede acceder a todas las características si se llaman mediante la sintaxis: nombreDelObjeto.caracteristica; Como se muestra en la última línea.

En el siguiente ejemplo se definirá dos funciones, la primera permitirá arrancar el carro, y la otra mostrara el estado en el que está el vehículo:

```
1 class Coche:
2     largoChasis = 200
3     anchorChasis = 100
4     ruedas = 4
5     enmarcha = False
6
7     def arrancar(self):
8         self.enmarcha = True
9
10    def estado(self):
11        if self.enmarcha:
12            return 'El coche se esta moviendo'
13        else:
14            return 'El coche esta quieto'
15
16    miCoche = Coche()
17    print(miCoche.estado())
18    miCoche.arrancar()
19    print(miCoche.estado())
```

En la condicional de la función estado, solo se puso el nombre de la variable, ya que, al ser de tipo booleano, por defecto va a verificar si esta es verdadera.

Ahora se puede crear otro objeto, y este obtendrá todas las características correspondientes a la clase, como ejercicio, crea otro objeto e imprime sus características, y por último verifica su estado, ya sea si el carro esa moviéndose o quieto.

En este momento todos los objetos pertenecientes a la clase `Coche` van a tener las mismas características, el problema con esto es que no todos los objetos son iguales, para permitir que al crear un objeto se le pongan las características

específicas se debe usar la función `__init__` la cual es una función conocida como el constructor, esta es una función normal, lo que hace que la clase la detecte como la principal es el nombre que se le pone (`__init__`) ya que esta es una palabra reservada.

A la función se le debe pasar como parámetro, la palabra `self`, junto a con los datos que son necesario pasarle, aunque esto también se puede usar para las demás funciones, por ejemplo:

```
1  class Coche:
2
3      def __init__(self, largoChasis, anchoChasis, ruedas):
4          self.largoChasis = largoChasis
5          self.anchoChasis = anchoChasis
6          self.ruedas = ruedas
7          self.enmarcha = False
8
9      def arrancar(self, enmarcha):
10         self.enmarcha = enmarcha
11         if self.enmarcha:
12             print('El coche se está moviendo')
13         else:
14             print('El coche está quieto')
15
16     def estado(self):
17         print('Largo del chasis:', self.largoChasis,
18             ',Ancho chasis:', self.anchoChasis,
19             ',Ruedas', self.ruedas)
20
21     miCoche = Coche(200,100,4)
22     miCoche.estado()
23
24     miCoche2 = Coche(400,200, 4)
25     miCoche2.estado()
```

En el código anterior, cada vez que se crea un objeto se debe definir las características que le pertenecen, también se puede ver que, aunque hay

variables que tienen el mismo nombre, el `self`, hace que se vuelvan variables diferentes.

La función `arrancar` ahora permite definir el valor de la variable booleana a la hora de llamarla, esto permite mover o dejar estático el coche, ya que la función que se tenía anteriormente solo permitía ponerlo en movimiento.

Si se quiere cambiar una característica en específico, se utiliza la siguiente sintaxis: `objeto.caracteristica = nuevoValor`; por ejemplo:

```
1  class Coche:
2
3      def __init__(self, largoChasis, anchoChasis, ruedas):
4          self.largoChasis = largoChasis
5          self.anchoChasis = anchoChasis
6          self.ruedas = ruedas
7          self.enmarcha = False
8
9      def arrancar(self, enmarcha):
10         self.enmarcha = enmarcha
11         if self.enmarcha:
12             print('El coche se está moviendo')
13         else:
14             print('El coche está quieto')
15
16     def estado(self):
17         print('Largo del chasis:', self.largoChasis,
18             ',Ancho chasis:', self.anchoChasis,
19             ',Ruedas', self.ruedas)
20
21 miCoche = Coche(200,100,4)
22 miCoche.estado()
23 miCoche.ruedas = 5
24 miCoche.estado()
```

En ciertas ocasiones, se necesita que alguna característica del objeto sea privada, es decir, que no se pueda acceder desde afuera, para esto se utiliza la técnica conocida como encapsulamiento, para encapsular un atributo se deben poner dos guiones bajos al comienzo del nombre del atributo, por ejemplo:

```
1  class Coche:
2
3      def __init__(self, largoChasis, anchoChasis, ruedas):
4          self.largoChasis = largoChasis
5          self.anchoChasis = anchoChasis
6          self.ruedas = ruedas
7          self.enmarcha = False
8          self.__numero_de_serie = 15181646816654
9
10     def arrancar(self, enmarcha):
11         self.enmarcha = enmarcha
12         if self.enmarcha:
13             print('El coche se está moviendo')
14         else:
15             print('El coche está quieto')
16
17     def estado(self):
18         print('Largo del chasis:', self.largoChasis,
19               ',Ancho chasis:', self.anchoChasis,
20               ',Ruedas', self.ruedas)
21
22 miCoche = Coche(200,100,4)
23 miCoche.estado()
24 print(miCoche.__numero_de_serie)
```

Al ejecutar el código anterior, se generará un error, debido a que, el encapsulamiento en el número de serie evita que desde afuera de la clase se pueda acceder a este atributo.

A continuación, veremos un ejemplo, con POO:

```
1 class Person:
2
3     def __init__(self, numero_de_identificacion, nombre, apellido, edad, fecha_nacimiento, direccion, telefono, email):
4         self._numero_de_identificacion = numero_de_identificacion
5         self.nombre = nombre
6         self.apellido = apellido
7         self.edad = edad
8         self.fecha_nacimiento = fecha_nacimiento
9         self.direccion = direccion
10        self.telefono = telefono
11        self.email = email
12
13    def datos(self):
14        print(
15            "Nombre:", self.nombre,
16            "Apellido:", self.apellido,
17            "Edad:", self.edad,
18            "Fecha de nacimiento:", self.fecha_nacimiento,
19            "Dirección:", self.direccion,
20            "Telefono:", self.telefono,
21            "Email:", self.email
22        )
23
24    person = Person(
25        123456,
26        'Jhon',
27        'Smith',
28        '41',
29        '02/01/1980',
30        'No. 12 Short Street, Greenville',
31        '555666',
32        'Jhon.smith@pythonCoder.com'
33    )
34
35    person.datos()
```


Herencia: La herencia permite que los atributos dentro de una clase se puedan reutilizar en una clase nueva, por ejemplo:

```
1 class Person:
2
3     def __init__(self, nombre, apellido, edad):
4         self.nombre = nombre
5         self.apellido = apellido
6         self.edad = edad
7
8     def datos(self):
9         print(
10             "Nombre:", self.nombre,
11             "Apellido:", self.apellido,
12             "Edad:", self.edad,
13         )
14
15 class Programador(Person):
16     def lenguaje_programacion(self):
17         print('Python')
18
19 daniel = Programador('Daniel', 'Alayón', 19)
20 daniel.datos()
21 daniel.lenguaje_programacion()
```

Como se puede ver a la clase programador se le pasa como parámetro la clase persona, esto hace que la clase programador obtenga todas las características y funciones de la clase persona, gracias a esto no es necesario volver a decirle que tiene una característica llamada nombre, solo toca poner el dato al crear un objeto.

La herencia se utiliza cuando algunos objetos tienen características en común pero también tienen características o métodos específicas de estos.

Si se vuelve a definir alguna función o atributo dentro de la clase que hereda las características, esta se sobrescribirá, lo que permite mayor control, por ejemplo:

```

1  class Person:
2
3      def __init__(self, nombre, apellido, edad):
4          self.nombre = nombre
5          self.apellido = apellido
6          self.edad = edad
7
8      def datos(self):
9          print(
10             "Nombre:", self.nombre,
11             "Apellido:", self.apellido,
12             "Edad:", self.edad,
13         )
14
15  class Programador(Person):
16
17      def __init__(self, nombre, apellido, edad):
18          Person.__init__(self, nombre, apellido, edad)
19          self.profesion = 'programador'
20
21      def lenguaje_programacion(self):
22          print('Python')
23
24  daniel = Programador('Daniel', 'Alayón', 19)
25  daniel.datos()
26  daniel.lenguaje_programacion()

```

En la línea 18 se llama al constructor de la clase persona, esto permite obtener los atributos de esta y agregar atributos extra como lo es la profesión, a esta técnica se le conoce como polimorfismo, ósea, la capacidad de cambiar de forma, sobrescribir una función heredada es una técnica de polimorfismo.

La herencia múltiple hace referencia a la posibilidad de que una clase herede atributos no solo de una clase anterior, sino, que puede heredar características de varias clases.

Métodos de cadenas (String)

En ciertos códigos las cadenas deben ser manipuladas, para esto tenemos algunos métodos que ya se encuentran en Python:

Nombre método	Función
Upper()	Pasa a mayúsculas un string
Lower()	Pasa a minúsculas un string
Capitalize()	Pone la primera letra en mayúscula
Count()	Cuenta cuantas veces aparece un char o un string dentro de un string
Find()	Encuentra el índice en el que aparece un string o un char dentro de un texto
Isdigit()	Verifica si el valor introducido es un número o no lo es
Isalunum()	Verifica si el valor introducido es alfanumérico

IsAlpha()	Comprueba si en el string existen solo letras
Split()	Separa las palabras de un texto
Strip()	Borra espacios que sobran al comienzo o al final de un texto
Replace()	Cambia una letra o una palabra por otra
Rfind()	Encuentra el índice en el que se encuentra un char o string pero buscando desde el final del texto

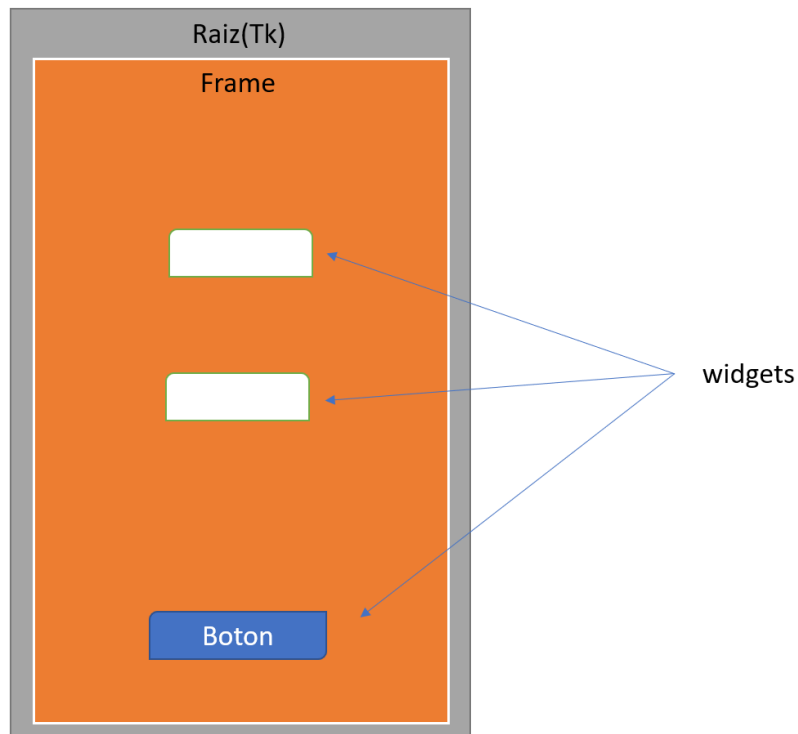
Interfaces graficas (GUI)

Hasta el momento se han venido escribiendo programas que funcionan en la consola de python, pero para presentarle un producto es mejor usar una gui (graphical user interface), ya que estas tienen como función, hacer de intermediario entre el usuario y el programa, una GUI esta conformada por botones, menús, casillas de verificación, etc.

Python tiene varias librerías que se pueden usar para crear interfaces, por ejemplo: tkinter, pyqt, PyGTK, entre otras; En la guía se va a utilizar la librería tkinter.

Tkinter funciona como conexión entre Python y la librería TCL/Tk la cual contiene los objetos que necesitamos para crear la interfaz.

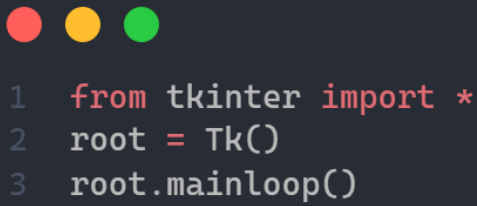
La estructura para crear una interfaz gráfica es la siguiente:



La raíz, es la ventana del programa en el cual se pondrán todos los elementos, aunque por buena organización lo mejor es utilizar un elemento conocido como el frame, ya que este permite organizar de una manera más sencilla los elementos de la interfaz.

Los widgets y objetos son los elementos que van a interactuar en la interfaz, botones, espacios para escribir texto, etc.

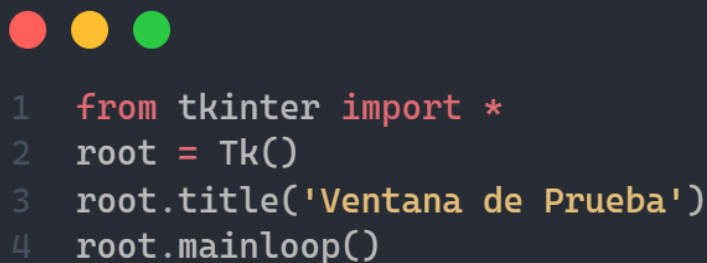
Creación ventana:



```
1 from tkinter import *
2 root = Tk()
3 root.mainloop()
```

Lo primero que se hace es importar todos los elementos de la Librería, seguido de esto se debe crear la raíz o ventana de la aplicación, para esto, se crea una variable y se iguala a Tk(), esto es para tener una variable en la que se puedan modificar todos los elementos de la ventana, la línea 3 genera un bucle infinito que evita que la ventana se cierre después de ejecutar el programa.

La variable root permite cambiarle ciertas configuraciones a la ventana principal, en el código anterior el nombre de la ventana es Tk pero con la variable root se puede cambiar, para esto, se pone el nombre de la variable seguido de un punto y luego la configuración que se quiera editar, en este caso el título:



```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.mainloop()
```

La ventana que se genera tiene un tamaño predeterminado, pero este se puede cambiar mediante el siguiente código:



```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.geometry('1280x720')
5 root.mainloop()
```

Los valores dentro del parámetro en la configuración geometry son dados en pixeles.

Hasta el momento el usuario ha tenido la posibilidad de cambiarle el tamaño a la ventana arrastrando uno de sus lados, el problema es que en ciertas ocasiones, se quiere que las ventanas tengan un tamaño específico y que el usuario no pueda cambiarlo, para esto se usa la opción de configuración resizable ya que esta permite definir si el usuario puede cambiar el ancho o al alto de la ventana, una gran ventaja de esta configuración es que se pueden utilizar valores diferentes para el alto y el ancho, a la configuración se le deben pasar dos parámetros el ancho y el alto, por ejemplo:



```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.geometry('1280x720')
5 root.resizable(True, False)
6 root.mainloop()
```

En el código anterior el ancho de la ventana se puede cambiar, pero el alto siempre se va a mantener igual.

Para crear un frame, se debe igualar una variable a la librería `Frame()`, pero esto solo crea el frame mas no le dice a donde va a pertenecer debido a esto se debe utilizar el método `pack()` el cual permite unir al frame con la raíz.

En los códigos anteriores se ha estado utilizando el método `geometry`, el problema con esto es que solo funciona con la raíz, y en un programa la raíz se debe adaptar al frame, es por esto que el tamaño se le debe poner al `Frame` mediante el método `config`.

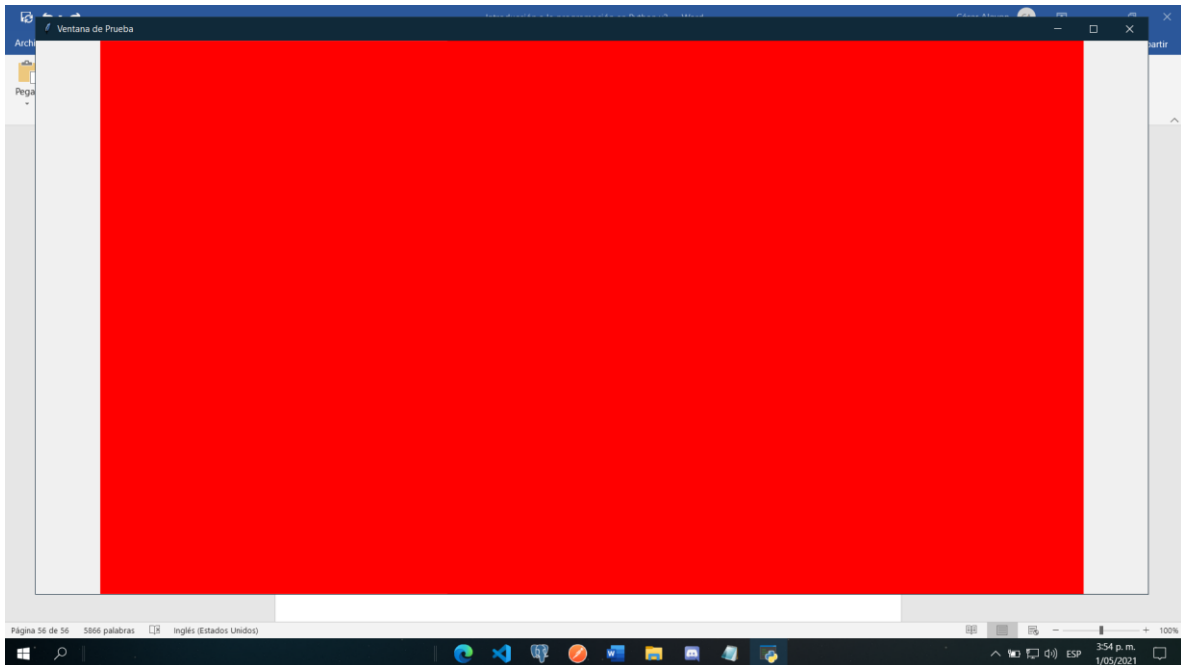
```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.resizable(True, False)
5
6 miFrame = Frame()
7 miFrame.pack()
8 miFrame.config(width = "1280", height = "720")
9
10 root.mainloop()
```

Con el código anterior obtenemos el mismo resultado que si se hiciera directamente con la raíz, con la diferencia que el contenido que se encuentra en la ventana es el frame y en el primer código era directamente la raíz, para comprobar esto se le puede cambiar el color al frame para ver donde está ubicado:

```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.resizable(True, False)
5
6 miFrame = Frame()
7 miFrame.pack()
8 miFrame.config(width = "1280", height = "720")
9 miFrame.config(bg = "red")
10 root.mainloop()
```


La ventana tendrá un color totalmente rojo, ya que la configuración bg hace referencia al background o fondo del frame.

Si se incrementa el ancho de la ventana veremos que queda un espacio que no es de color rojo:



Esto sucede debido a que aunque la raíz se adapta al frame, el frame no se adapta a la raíz, para solucionar esto se debe poner la opción fill a la hora de darle el pack al frame, fill tiene varias opciones, adaptar el eje x, el eje y o ambos, en el caso de que se quiera un eje en específico se debe poner la letra del eje, si se quieren ambos se debe poner la palabra both, en el código de ejemplo se quiere que ambos lados se puedan estirar, pero para que se pueda estirar el eje y se debe poner la configuración expand en true, esto hará que al cambiar el tamaño de la ventana el Frame también lo haga:

```

1  from tkinter import *
2  root = Tk()
3  root.title('Ventana de Prueba')
4  root.resizable(True, True)
5
6  miFrame = Frame()
7  miFrame.pack(fill="both", expand="true")
8  miFrame.config(width = "1280", height = "720")
9  miFrame.config(bg = "red")
10 root.mainloop()

```

Widget(objeto):

La mayoría de objetos tienen la misma sintaxis:

nombre_variable = nombreWidget(contenedor, opciones)

El contenedor es básicamente a qué frame va a pertenecer nuestro widget, las opciones pueden ser diferentes dependiendo del objeto, en la guía se van a usar solo unas cuantas de estas opciones.

Label: Permite mostrar texto o imágenes en el contenedor

```

1  from tkinter import *
2  root = Tk()
3  root.title('Ventana de Prueba')
4  root.resizable(True, True)
5
6  miFrame = Frame(root, width = 500, height = 400)
7  miFrame.pack()
8
9  miLabel = Label(miFrame, text = "Hola Mundo")
10 miLabel.place(x = 100, y = 200)
11
12 root.mainloop()

```

El método `place` va a ubicar el label en la posición (x, y) que se le pase como parámetro.

Para cambiar el color de texto y la fuente se utilizan los parámetros `fg` y `Font` respectivamente, el parámetro `Font` permite no solo cambiar la fuente del texto sino también cambiar su tamaño en pixeles.

```
1 from tkinter import *
2 root = Tk()
3 root.title('Ventana de Prueba')
4 root.resizable(True, True)
5
6 miFrame = Frame(root, width = 500, height = 400)
7 miFrame.pack()
8
9 miLabel = Label(miFrame, text = "Hola Mundo", fg = "red", font = ("Comic Sans MS", 18))
10 miLabel.place(x = 100, y = 200)
11
12 root.mainloop()
```

Entry: Es un cuadro que permite la entrada de texto

Por comodidad y buena practica, es recomendable usar la opcion `grid` para ubicar los elementos en un frame, ya que se vuelve mas sencillo, la function `grid` genera una table (No visible) en el frame, y los elementos que se van a poner en el frame se ubican en una de las casillas.

```
entryNombre = Entry(miFrame)
entryNombre.grid(row = 0, column = 2)
```

```

1  from tkinter import *
2  root = Tk()
3  root.title('Ventana de Prueba')
4  root.resizable(True, True)
5
6  miFrame = Frame(root, width = 500, height = 400)
7  miFrame.pack()
8
9  miLabel = Label(miFrame, text = "Nombre: ", font = ("Arial", 12))
10 miLabel.grid(row = 0, column = 1)
11
12 entryNombre = Entry(miFrame)
13 entryNombre.grid(row = 0, column = 2)
14
15
16 root.mainloop()

```

El anterior código muestra un entry en el cual se debe escribir el nombre de una persona, el label y el entry se van a mostrar en la misma fila pero en una columna diferente.

Al ejecutar el código se evidencia que al utilizar la opción grid el frame se comienza a la cantidad de columnas y filas que se están generando, por esta razón aunque se le establezca ese tamaño, la Ventana se verá más pequeña, para evitar esto se puede utilizar una opción llamada padding cuya función es separar a los elementos, el padding se maneja de manera separada para los ejes (x,y), para utilizar esta función se debe poner padx en el caso del eje x y pady en el eje y.



```

1  from tkinter import *
2  root = Tk()
3  root.title('Ventana de Prueba')
4  root.resizable(True, True)
5
6  miFrame = Frame(root, width = 500, height = 400)
7  miFrame.pack()
8
9  label_nombre = Label(miFrame, text = "Nombre: ", font = ("Arial", 10))
10 label_nombre.grid(row = 0, column = 1, padx = 15, pady = 10)
11
12 entry_nombre = Entry(miFrame)
13 entry_nombre.grid(row = 0, column = 2, padx = 15, pady = 10)
14
15
16 label_apellido = Label(miFrame, text = "Apellido: ", font = ("Arial", 10))
17 label_apellido.grid(row = 1, column = 1, padx = 15, pady = 10)
18
19 entry_apellido = Entry(miFrame)
20 entry_apellido.grid(row = 1, column = 2, padx = 15, pady = 10)
21
22 root.mainloop()

```

Boton: Los botones son widget que permiten ejecutar acciones, para que estas funcionen por lo general se conectan a una funcion mediante una opcion que permite ejecutar una funcion lambda

Para ver todos los elementos anteriores funcionando, realizaremos como practica una calculadora:



```

1  from tkinter import *
2
3  raiz=Tk()
4
5  miFrame=Frame(raiz)
6
7  miFrame.pack()
8
9  operacion=""
10
11 reset_pantalla=False
12
13 resultado=0
14

```

Lo primero es importar la librería tkinter junto a todos sus componentes, luego se crea la raíz, creamos el frame principal, y al final unas variables que se van a necesitar más adelante.

```
1 #-----pantalla-----
2
3 numeroPantalla=StringVar()
4
5 pantalla=Entry(miFrame, textvariable=numeroPantalla)
6 pantalla.grid(row=1, column=1, padx=10, pady=10, columnspan=4)
7 pantalla.config(background="black", fg="#03f943", justify="right")
```

Esta parte del Código se encarga de crear la pantalla de la calculadora, en este caso se tienen unos conceptos nuevos.

StringVar: Le dice a la variable que va a almacenar un string, pero la variable queda vacía.

Columnspan: Se encarga de hacer que un widget pueda ocupar mas de una sola fila

Justify: Se encarga de decir hacia donde va a estar justificado el texto, en este caso la derecha.

Fg: fg es el color del texto, pero en este caso usamos un color hexadecimal, esto nos permite tener exactamente el color que queremos.

TextVariable: Hace referencia a la variable que va a estar ligada el entry que va a funcionar como la pantalla de la calculadora, esta va a almacenar los datos.

```

1  #-----pulsaciones teclado-----
2
3  def numeroPulsado(num):
4
5      global operacion
6
7      global reset_pantalla
8
9      if reset_pantalla!=False:
10
11         numeroPantalla.set(num)
12
13         reset_pantalla=False
14
15     else:
16
17         numeroPantalla.set(numeroPantalla.get() + num)

```

Esta función se encarga de detectar el número que se está pulsando y lo pone en la pantalla, para esto primero detecta si la pantalla esta vacía, de ser así, simplemente pone el número en la pantalla, si ya hay un número o más en la pantalla, toma lo que está en la está, le agrega el nuevo número y lo muestra en pantalla.

Las variables globales, son variables a las cuales se puede acceder en cualquier momento, esto en ciertas ocasiones puede evitar errores, aunque no siempre se deben usar ya que se podrían editar por accidente más adelante, en este código se van a usar variables globales ya que varias funciones las van a necesitar en alguna parte del código.

A partir de aquí se pondrán las funciones que se encargan de realizar las operaciones matemáticas.

```
1 #-----funcion suma-----
2
3 def suma(num):
4
5     global operacion
6
7     global resultado
8
9     global reset_pantalla
10
11     resultado+=int(num) #resultado=resultado+int(num)
12
13     operacion="suma"
14
15     reset_pantalla=True
16
17     numeroPantalla.set(resultado)
```

Se encarga de sumar el número que estaba en la pantalla con el número nuevo, también pone en true el reset de la pantalla para que cuando se pulse otra vez un número la función numeroPulsado, borre lo que esta en la pantalla y comience a escribir la nueva operación, finalmente muestra el resultado en pantalla.


```

1  #-----funcion resta-----
2  num1=0
3
4  contador_resta=0
5
6  def resta(num):
7
8      global operacion
9
10     global resultado
11
12     global num1
13
14     global contador_resta
15
16     global reset_pantalla
17
18     if contador_resta==0:
19
20         num1=int(num)
21
22         resultado=num1
23
24     else:
25
26         if contador_resta==1:
27
28             resultado=num1-int(num)
29
30         else:
31
32             resultado=int(resultado)-int(num)
33
34         numeroPantalla.set(resultado)
35
36         resultado=numeroPantalla.get()
37
38
39     contador_resta=contador_resta+1
40
41     operacion="resta"
42
43     reset_pantalla=True

```

```

1  #-----funcion multiplicacion-----
2  contador_multi=0
3
4  def multiplica(num):
5
6      global operacion
7
8      global resultado
9
10     global num1
11
12     global contador_multi
13
14     global reset_pantalla
15
16     if contador_multi==0:
17
18         num1=int(num)
19
20         resultado=num1
21
22     else:
23
24         if contador_multi==1:
25
26             resultado=num1*int(num)
27
28         else:
29
30             resultado=int(resultado)*int(num)
31
32             numeroPantalla.set(resultado)
33
34             resultado=numeroPantalla.get()
35
36
37     contador_multi=contador_multi+1
38
39     operacion="multiplicacion"
40
41     reset_pantalla=True

```

```

1  #-----funcion division-----
2
3  contador_divi=0
4
5  def divide(num):
6
7      global operacion
8
9      global resultado
10
11     global num1
12
13     global contador_divi
14
15     global reset_pantalla
16
17     if contador_divi==0:
18
19         num1=float(num)
20
21         resultado=num1
22
23     else:
24
25         if contador_divi==1:
26
27             resultado=num1/float(num)
28
29         else:
30
31             resultado=float(resultado)/float(num)
32
33             numeroPantalla.set(resultado)
34
35             resultado=numeroPantalla.get()
36
37
38     contador_divi=contador_divi+1
39
40     operacion="division"
41
42     reset_pantalla=True

```

```

1  #-----funcion el_resultado-----
2
3  def el_resultado():
4
5      global resultado
6
7      global operacion
8
9      global contador_resta
10
11     global contador_multi
12
13     global contador_divi
14
15
16     if operacion=="suma":
17
18         numeroPantalla.set(resultado+int(numeroPantalla.get()))
19
20         resultado=0
21
22     elif operacion=="resta":
23
24         numeroPantalla.set(int(resultado)-int(numeroPantalla.get()))
25
26         resultado=0
27
28         contador_resta=0
29
30     elif operacion=="multiplicacion":
31
32         numeroPantalla.set(int(resultado)*int(numeroPantalla.get()))
33
34         resultado=0
35
36         contador_multi=0
37
38     elif operacion=="division":
39
40         numeroPantalla.set(int(resultado)/int(numeroPantalla.get()))
41
42         resultado=0
43
44         contador_divi=0

```

```

1 #-----fila 1-----
2
3 boton7=Button(miFrame, text="7", width=3, command=lambda:numeroPulsado("7"))
4 boton7.grid(row=2, column=1)
5 boton8=Button(miFrame, text="8", width=3, command=lambda:numeroPulsado("8"))
6 boton8.grid(row=2, column=2)
7 boton9=Button(miFrame, text="9", width=3, command=lambda:numeroPulsado("9"))
8 boton9.grid(row=2, column=3)
9 botonDiv=Button(miFrame, text="/", width=3, command=lambda:divide(numeroPantalla.get()))
10 botonDiv.grid(row=2, column=4)
11

```

El código de las widgets se van a organizar por filas para mayor comodidad, la última opción que se encuentra en los botones se encarga de decirle al botón que función va a ejecutar, para esto se utiliza la opción `command` el cual ejecuta una función `lambda` que se encarga de llamar a la función.

```

1 #-----fila 2-----
2
3 boton4=Button(miFrame, text="4", width=3, command=lambda:numeroPulsado("4"))
4 boton4.grid(row=3, column=1)
5 boton5=Button(miFrame, text="5", width=3, command=lambda:numeroPulsado("5"))
6 boton5.grid(row=3, column=2)
7 boton6=Button(miFrame, text="6", width=3, command=lambda:numeroPulsado("6"))
8 boton6.grid(row=3, column=3)
9 botonMult=Button(miFrame, text="x", width=3, command=lambda:multiplifica(numeroPantalla.get()))
10 botonMult.grid(row=3, column=4)

```

```

1 #-----fila 3-----
2
3 boton1=Button(miFrame, text="1", width=3, command=lambda:numeroPulsado("1"))
4 boton1.grid(row=4, column=1)
5 boton2=Button(miFrame, text="2", width=3, command=lambda:numeroPulsado("2"))
6 boton2.grid(row=4, column=2)
7 boton3=Button(miFrame, text="3", width=3, command=lambda:numeroPulsado("3"))
8 boton3.grid(row=4, column=3)
9 botonRest=Button(miFrame, text="-", width=3, command=lambda:resta(numeroPantalla.get()))
10 botonRest.grid(row=4, column=4)

```

```
1 #-----fila 4-----
2
3 boton0=Button(miFrame, text="0", width=3, command=lambda:numeroPulsado("0"))
4 boton0.grid(row=5, column=1)
5 botonComa=Button(miFrame, text=",", width=3, command=lambda:numeroPulsado("."))
6 botonComa.grid(row=5, column=2)
7 botonIgual=Button(miFrame, text="=", width=3, command=lambda:el_resultado())
8 botonIgual.grid(row=5, column=3)
9 botonSum=Button(miFrame, text="+", width=3, command=lambda:suma(numeroPantalla.get()))
10 botonSum.grid(row=5, column=4)
11
12 raiz.mainloop()
13
```